Relatório de Execução

Projeto: Desafio 2 - Serverest Compass Uol

Testador: Juan Castro Mendonça de Oliveira

Data: 09/02/2026

Ambiente: Windows 11 | Firefox | Rede Wifi 5g | Postman Web

---

1. Execução- Cenários de Teste

Execução dos cenários de teste descritos no Plano de Testes pelo Postman no browser Firefox.

TL-01

Cenário: Realizar login com dados válidos e usuário existente.

Endpoint: /login (POST).

Resultado esperado: A aplicação deve permitir o acesso e retornar o token de autenticação.

Resultado do teste: Conforme o esperado.

TL-02

Cenário: Realizar login com dados inválidos.

Endpoint: /login (POST).

Resultado esperado: A aplicação deve retornar um erro 400 indicando que os dados devem ser preenchidos corretamente.

Resultado do teste: Conforme o esperado.

TL-03

Cenário: Realizar login com email não cadastrado.

Endpoint: /login (POST).

Resultado esperado: A aplicação deve retornar um erro indicando que não há acesso para esse email.

Resultado do teste: Conforme o esperado.

TU-01

Cenário: Listar usuários.

Endpoint: /usuarios (GET).

Resultado esperado: Retornar a lista de usuários.

Resultado do teste: Conforme o esperado.

TU-02

Cenário: Cadastrar usuários com dados válidos.

Endpoint: /usuarios (POST).

Resultado esperado: Cadastrar usuário com sucesso e retornar o status 201.

Resultado do teste: Conforme o esperado.

TU-03

Cenário: Cadastrar usuários com dados inválidos.

Endpoint: /usuarios (POST).

Resultado esperado: Retornar um erro 400 indicando que os dados foram preenchidos incorretamente.

Resultado do teste: Conforme o esperado.

TU-04

Cenário: Cadastrar usuário com email já cadastrado.

Endpoint: /usuarios (POST).

Resultado esperado: Retornar um erro indicando que o email inserido já está sendo utilizado.

Resultado do teste: Conforme o esperado.

TU-05

Cenário: Buscar um usuário válido e existente.

Endpoint: /usuarios/{id} (GET).

Resultado esperado: Retornar o usuário correspondente com o status 200.

Resultado do teste: Conforme o esperado.

TU-06

Cenário: Buscar um usuário com ID inválido.

Endpoint: /usuarios/{id} (GET).

Resultado esperado: Retornar um erro 400 indicando que o formato do ID está incorreto.

Resultado do teste: Conforme o esperado.

TU-07

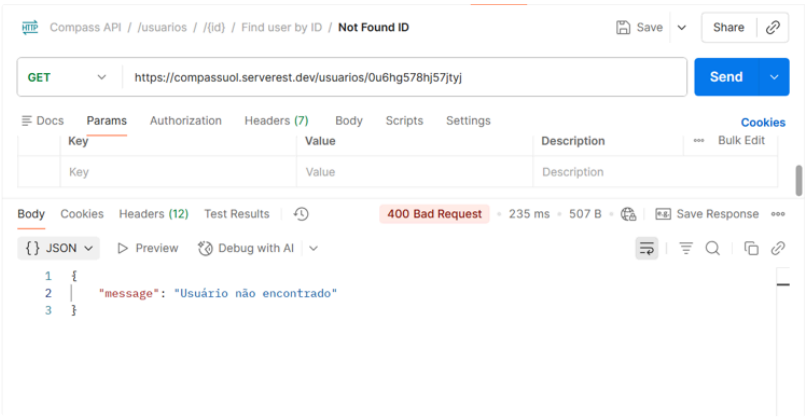
Cenário: Buscar um usuário com ID inexistente.

Endpoint: /usuarios/{id} (GET).

Resultado esperado: Retornar um erro 404 indicando que o ID não foi encontrado.

Resultado do teste: Tratamento inadequado, a aplicação deveria retornar um erro 404 (Not Found).

Evidência:



TU-08

Cenário: Excluir um usuário válido e existente.

Endpoint: /usuarios/{id} (DELETE).

Resultado esperado: Retornar um código 200 e excluir o usuário.

Resultado do teste: Conforme o esperado.

TU-09

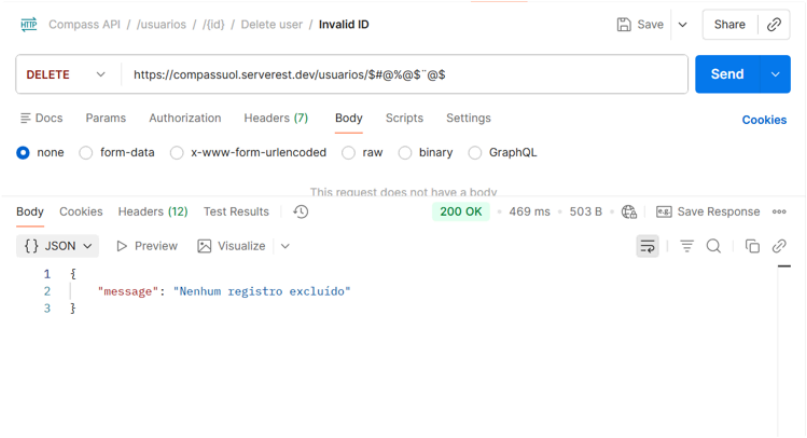
Cenário: Excluir um usuário com ID inválido.

Endpoint: /usuarios/{id} (DELETE).

Resultado esperado: Retornar um uma mensagem de erro com o status 400 indicando que o formato do ID é inválido.

Resultado do teste: Tratamento inadequado, a aplicação retorna um status de sucesso.

Evidência:



TU-10

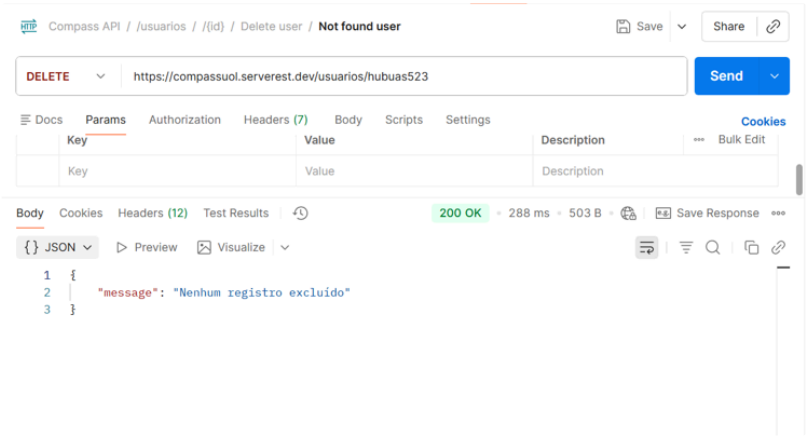
Cenário: Excluir um usuário com ID inexistente.

Endpoint: /usuarios/{id} (DELETE).

Resultado esperado: Retornar um uma mensagem de erro com o status 404 indicando que o usuário não foi encontrado.

Resultado do teste: Tratamento inadequado, a aplicação retorna um status de sucesso.

Evidência:



TU-11

Cenário: Atualizar um usuário inserindo dados válidos e um ID existente.

Endpoint: /usuarios/{id} (PUT).

Resultado esperado: Atualizar o usuário e retornar um status 200.

Resultado do teste: Conforme o esperado.

TU-12

Cenário: Criar um usuário inserindo dados válidos e um ID inexistente.

Endpoint: /usuarios/{id} (PUT).

Resultado esperado: Criar o novo usuário e retornar um status 201.

Resultado do teste: Conforme o esperado.

TU-13

Cenário: Atualizar ou criar um usuário com dados inválidos.

Endpoint: /usuarios/{id} (PUT).

Resultado esperado: Retornar um erro com o status 400 indicando que os dados inseridos são inválidos.

Resultado do teste: Conforme o esperado.

TU-14

Cenário: Criar um usuário com ID inexistente e email já utilizado.

Endpoint: /usuarios/{id} (PUT).

Resultado esperado: Retornar um erro indicando que o email inserido já está sendo utilizado.

Resultado do teste: Conforme o esperado.

TP-01

Cenário: Listar produtos.

Endpoint: /produtos (GET).

Resultado esperado: Retornar a lista de produtos.

Resultado do teste: Conforme o esperado.

TP-02

Cenário: Cadastrar produto com dados válidos.

Endpoint: /produtos (POST).

Resultado esperado: Cadastrar produto com sucesso e retornar o status 201.

Resultado do teste: Conforme o esperado.

TP-03

Cenário: Cadastrar produto com dados inválidos.

Endpoint: /produtos (POST).

Resultado esperado: Retornar um erro 400 indicando que os dados foram preenchidos incorretamente.

Resultado do teste: Conforme o esperado.

TP-04

Cenário: Cadastrar produto com nome já utilizado.

Endpoint: /usuarios (POST).

Resultado esperado: Retornar um erro indicando que o nome inserido já está sendo utilizado.

Resultado do teste: Conforme o esperado.

TP-05

Cenário: Buscar um produto válido e existente.

Endpoint: /produtos/{id} (GET).

Resultado esperado: Retornar o produto correspondente com o status 200.

Resultado do teste: Conforme o esperado.

TP-06

Cenário: Buscar um produto com ID inválido.

Endpoint: /produtos/{id} (GET).

Resultado esperado: Retornar um erro 400 indicando que o formato do ID está incorreto.

Resultado do teste: Conforme o esperado.

TP-07

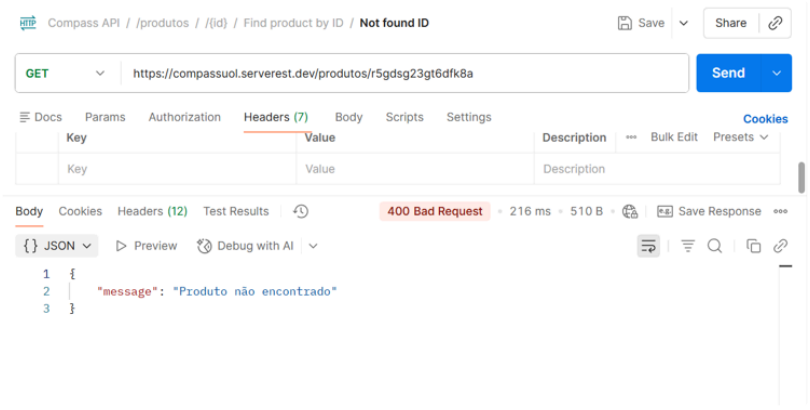
Cenário: Buscar um produto com ID inexistente.

Endpoint: /produtos/{id} (GET).

Resultado esperado: Retornar um erro 404 indicando que o ID não foi encontrado.

Resultado do teste: Tratamento inadequado, a aplicação deveria retornar um erro 404 (Not Found).

Evidência:



TP-08

Cenário: Excluir um produto válido e existente.

Endpoint: /produtos/{id} (DELETE).

Resultado esperado: Retornar um código 200 e excluir o produto.

Resultado do teste: Conforme o esperado.

TP-09

Cenário: Excluir um produto com ID inválido.

Endpoint: /produtos/{id} (DELETE).

Resultado esperado: Retornar um uma mensagem de erro com o status 400 indicando que o formato do ID é inválido.

Resultado do teste: Conforme o esperado.

TP-10

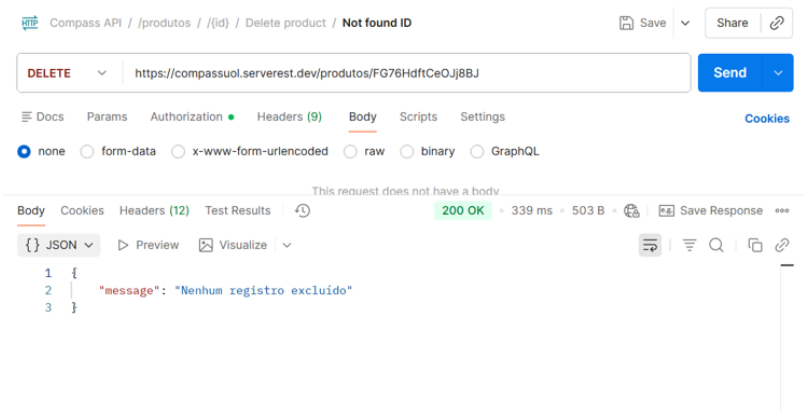
Cenário: Excluir um produto com ID inexistente.

Endpoint: /produtos/{id} (DELETE).

Resultado esperado: Retornar um uma mensagem de erro com o status 404 indicando que o produto não foi encontrado.

Resultado do teste: Tratamento inadequado, a aplicação retorna um status de sucesso.

Evidência:



TU-11

Cenário: Atualizar um produto inserindo dados válidos e um ID existente.

Endpoint: /produto/{id} (PUT).

Resultado esperado: Atualizar o produto e retornar um status 200.

Resultado do teste: Conforme o esperado.

TP-12

Cenário: Criar um produto inserindo dados válidos e um ID inexistente.

Endpoint: /produtos/{id} (PUT).

Resultado esperado: Criar o novo produto e retornar um status 201.

Resultado do teste: Conforme o esperado.

TP-13

Cenário: Atualizar ou criar um produto com dados inválidos.

Endpoint: /produtos/{id} (PUT).

Resultado esperado: Retornar um erro com o status 400 indicando que os dados inseridos são inválidos.

Resultado do teste: Conforme o esperado.

TP-14

Cenário: Criar um produto com ID inexistente e nome já utilizado.

Endpoint: /produtos/{id} (PUT).

Resultado esperado: Retornar um erro indicando que o nome inserido já está sendo utilizado.

Resultado do teste: Conforme o esperado.

TC-01

Cenário: Listar carrinhos.

Endpoint: /carrinhos (GET).

Resultado esperado: Retornar a lista de carrinhos.

Resultado do teste: Conforme o esperado.

TC-02

Cenário: Cadastrar carrinho com dados válidos.

Endpoint: /carrinhos (POST).

Resultado esperado: Cadastrar carrinho com sucesso e retornar o status 201.

Resultado do teste: Conforme o esperado.

TC-03

Cenário: Cadastrar carrinho com dados válidos e um token inválido.

Endpoint: /carrinhos (POST).

Resultado esperado: Retornar um erro 401 de falha de autorização.

Resultado do teste: Conforme o esperado.

TC-04

Cenário: Cadastrar carrinho com dados inválidos.

Endpoint: /carrinhos (POST).

Resultado esperado: Retornar um erro 400 informando que os dados inseridos são inválidos.

Resultado do teste: Conforme o esperado.

TC-05

Cenário: Cadastrar carrinho com um token de usuário já vinculado a outro carrinho.

Endpoint: /carrinhos (POST).

Resultado esperado: Retornar um erro informando que o usuário só pode estar vinculado a 1 carrinho por vez.

Resultado do teste: Conforme o esperado.

TC-06

Cenário: Buscar carrinho com um ID válido e existente.

Endpoint: /carrinhos/{id} (GET).

Resultado esperado: Retornar o carrinho correspondente e o status 200.

Resultado do teste: Conforme o esperado.

TC-07

Cenário: Buscar um carrinho com ID inválido.

Endpoint: /carrinhos/{id} (GET).

Resultado esperado: Retornar um erro 400 indicando que o formato do ID está incorreto.

Resultado do teste: Conforme o esperado.

TC-08

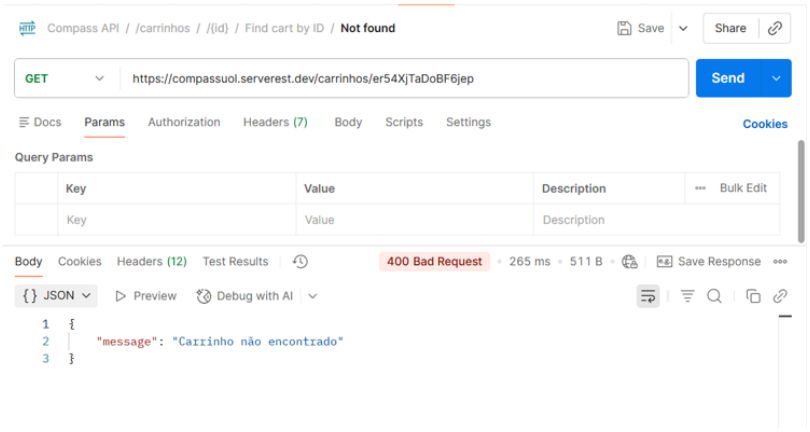
Cenário: Buscar um carrinho com ID inexistente.

Endpoint: /carrinhos/{id} (GET).

Resultado esperado: Retornar um erro 404 indicando que o carrinho não foi encontrado.

Resultado do teste: Tratamento inadequado, é retornado um erro 400.

Evidência:



TC-09

Cenário: Concluir compra com token válido.

Endpoint: /carrinhos/concluir-compra (DELETE).

Resultado esperado: Retornar um status 200 e excluir o carrinho vinculado ao token.

Resultado do teste: Conforme o esperado.

TC-10

Cenário: Concluir compra sem um token válido.

Endpoint: /carrinhos/concluir-compra (DELETE).

Resultado esperado: Retornar uma mensagem de erro de autorização com o status 401.

Resultado do teste: Conforme o esperado.

TC-11

Cenário: Cancelar compra com um token válido.

Endpoint: /carrinhos/cancelar-compra (DELETE).

Resultado esperado: Retornar um status 200 de sucesso, excluir o carrinho e retornar a quantidade de itens do carrinho de volta ao estoque.

Resultado do teste: Conforme o esperado.

TC-12

Cenário: Cancelar compra sem um token válido.

Endpoint: /carrinhos/cancelar-compra (DELETE).

Resultado esperado: Retornar uma mensagem de erro de autorização com o status 401.

Resultado do teste: Conforme o esperado.

2. **Conclusão - Cenários de Teste**

Com a execução dos cenários de teste a aplicação se provou bastante consistente e com as devidas regras de negócio expressas no ServeRest aplicadas, sem bugs identificados, porém com algumas recomendações de correção no tratamento de falhas, sendo elas:

- Alterar o status code das requisições GET de 400 para 404 em casos de objetos não encontrados;
- Alterar o status code das requisições DELETE de 200 para 404 em casos de objetos não encontrados e emitir uma mensagem de erro informando que o ID não foi localizado;
- Alterar o status code das requisições DELETE de 200 para 400 em casos de IDs inválidos e emitir uma mensagem de erro informando que o ID inserido está em um formato inválido.

2. **Execução - Testes exploratórios**

I- Criação do usuário e login

- **Tentativas de sabotagem:**

1. Tentar criar usuário com o campo “administrador“ diferente de ‘true’ ou ‘false’;
2. Tentativa de passar um email com formato incorreto.

- **Resultado do teste:**

1. ✅ Tratamento adequado de falhas, sem problemas identificados.

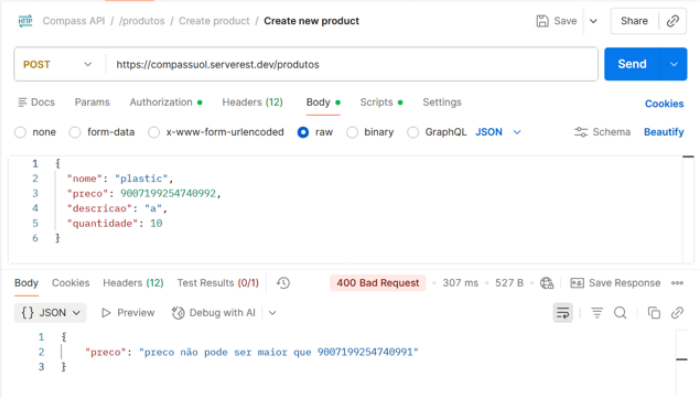
II- Criação e exclusão de um produto

- **Tentativas de sabotagem:**

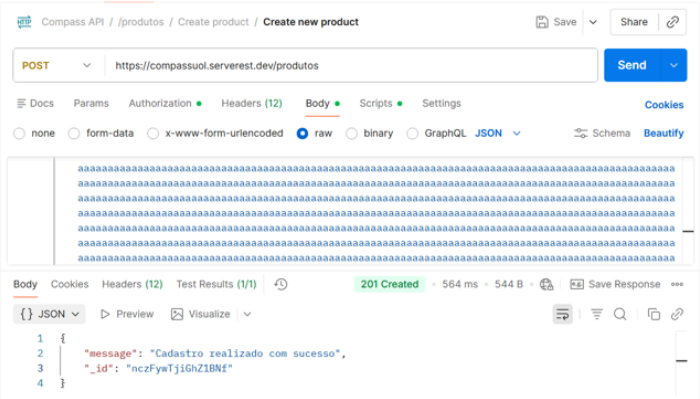
1. Tentativa de criar um produto sem autorização de administrador;
2. Criação de um produto com a descrição extremamente extensa;
3. Criação de um produto com o preço extremamente alto;
4. Tentar excluir um produto que faz parte de um carrinho.

- **Resultados do teste:**

1. ✅ Tratamento correto no caso de falta de autorização ;
2. ⚠️ O teste de criação com preço extremamente alto não admite valores superiores a `9007199254740991`, que já é um valor extremamente alto:



3. ⚠️ A aplicação não tem um limite de caracteres para o nome e descrição do produto:



4. ✅ Tratamento correto no caso de excluir um produto que já faz parte de um carrinho.

III- Criação e exclusão de um carrinho

- **Tentativas de sabotagem:** Inserir mais produtos no carrinho que o disponível em estoque; Inserir uma quantidade negativa de produtos no carrinho; Excluir um carrinho de um usuário sem carrinho vinculado.
- **Resultado do teste:** ✅ Tratamento adequado de falhas, sem problemas identificados.

IV- Edição de um produto e verificação de carrinhos

- **Tentativas de sabotagem:**

1. Deixar o produto com preço igual a 0;
2. Deixar o produto sem descrição;
3. Buscar carrinhos ou produtos com parâmetros inválidos.
4. Valor total do carrinho acima do máximo dos produtos.

- **Resultado do teste:** ✅ Tratamento adequado de falhas, sem problemas identificados.

3. **Conclusão - Testes Exploratórios**

Com a execução dos testes exploratórios baseados nas técnicas de Tour do guia turístico (Percorrendo as principais funcionalidades) e Tour do sabotador (Focando na tentativa de quebrar a aplicação), foi possível observar ainda mais que a aplicação é livre de problemas críticos e tem um bom tratamento de falhas, apenas com as seguintes ressalvas:

- A aplicação permite a inserção de valores extremamente elevados para os produtos;
- A aplicação não possui um limite de caracteres nos campos do tipo string dos produtos.

4. **Report de Issues**

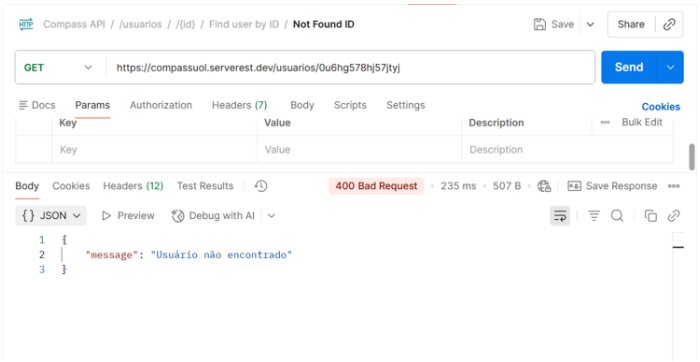
I- Ao buscar pelo ID um objeto não encontrado é retornado um status code 400 ao invéz de 404.

Passo a Passo: Em uma requisição do tipo /{objeto}/{id} com o verbo GET, passe um parâmetro ID válido porém não registrado. Será retornada uma mensagem indicando que o objeto não foi encontrado, porém com o status code 400.

Resultado Esperado: A aplicação deveria retornar um status code 404 (Not Found).

Prioridade: Baixa.

Evidência:



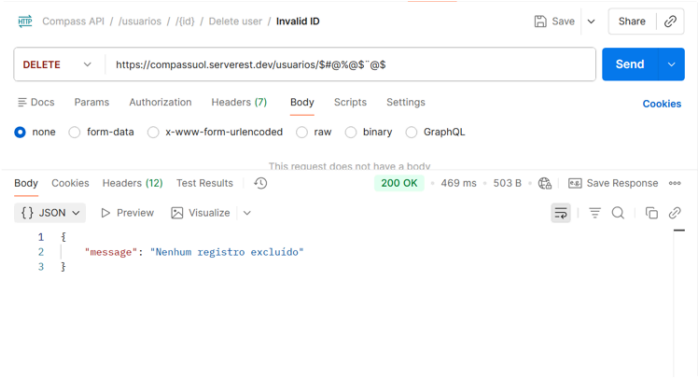
II- Ao enviar uma requisição DELETE com ID inválido ou não localizado é retornado um status de sucesso.

Passo a Passo: Em uma requisição do tipo /{objeto}/{id} com o verbo DELETE, passe um parâmetro ID válido ou inexistente. Será retornada uma mensagem indicando que nenhum objeto foi excluído e um status code de sucesso (200).

Resultado Esperado: A aplicação deveria retornar um status code 404 (Not Found) quando o ID não for encontrado e 400 (Bad Request) quando for passado um ID de formato inválido.

Prioridade: Média.

Evidência:



III- A aplicação aceita preços extremamente altos no criação ou edição de um produto.

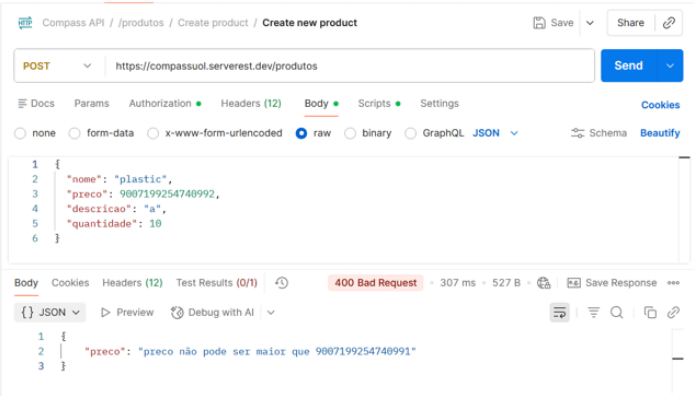
Passo a Passo: No corpo de uma requisição de endpoint "/produtos" com o verbo POST ou endpoint "/produtos/{id}" de verbo "PUT", insira no campo "precos" qualquer valor muito alto até nove quatrilhões sete trilhões cento e noventa e nove bilhões duzentos e cinquenta e quatro milhões setecentos e quarenta mil novecentos e noventa e um reais e setenta e dois centavos.

A aplicação aceitará e criará ou editará o produto inserindo o preço.

Resultado Esperado: A aplicação deveria ter um limite menor de preço máximo, considerando que é um e-commerce o valor de cem mil seria mais aceitável.

Prioridade: Média.

Evidência:



IV- A aplicação não possui um limite de caracteres nos campos do tipo string ao criar ou editar um produto.

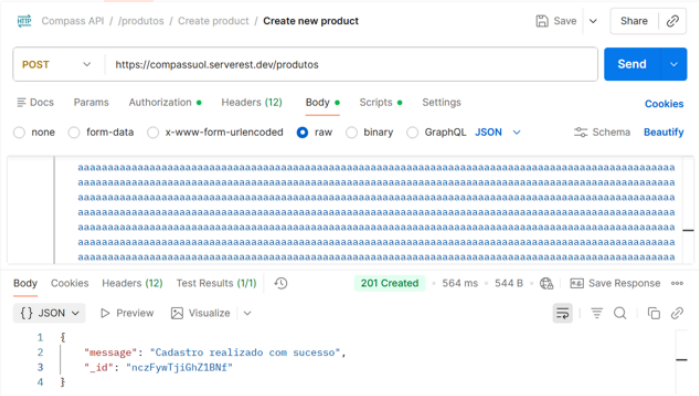
Passo a Passo: No corpo de uma requisição de endpoint "/produtos" com o verbo POST ou endpoint "/produtos/{id}" de verbo "PUT", insira em um campo do tipo string um número extremamente grande de caracteres (mil por exemplo).

A aplicação aceitará e criará ou editará o produto inserindo o valor passado pelo usuário.

Resultado Esperado: A aplicação deveria ter um limite de caracteres mais restrito para o campo "nome" e um pouco mais extenso para o campo "descricao".

Prioridade: Baixa.

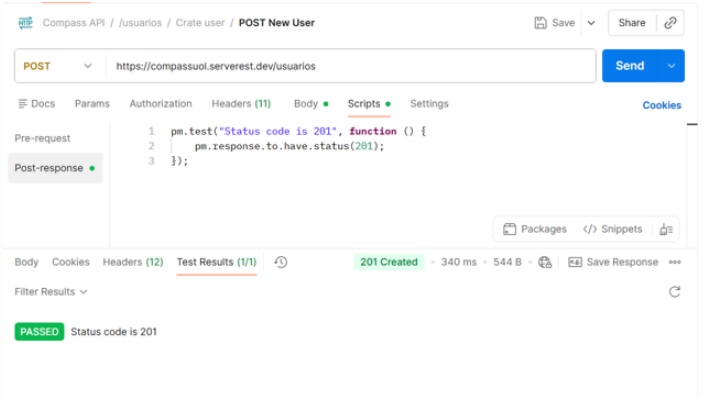
Evidência:



5. **Scripts criados no postman**

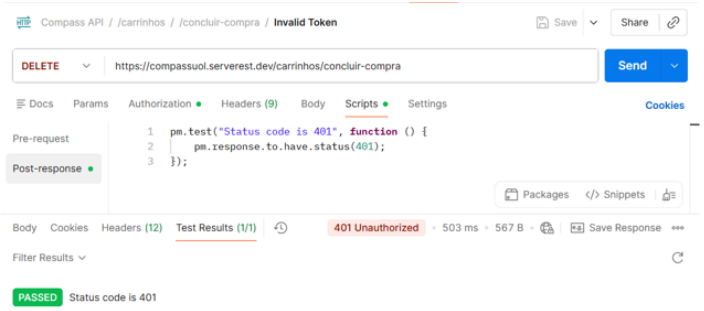
- **Validação de status code para casos válidos.**

  Exemplo:



- **Validação de status code para casos inválidos.**

  Exemplo:

- **Validação de status code para regras de negócio.**

  Exemplo: